

SYSTEME DIDACTIQUE POUR L'EXPERIMENTATION DU RESEAU C.A.N.

***Véhicule
Multiplexé
Didactique
VMD 01***



DIDALAB
5 Rue du Groupe Manoukian
78990 Elancourt
Tel: 01.30.66.08.88 / Fax: 01.30.66.72.20
ge@didalab.fr

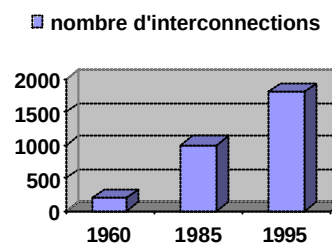
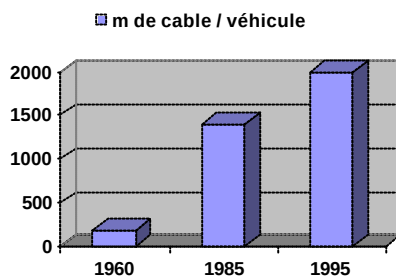
SOMMAIRE

1	GENERALITES	5
2	LA DISTRIBUTION D'ENERGIE	7
2.1	INFLUENCE DU « MULTIPLEXAGE » SUR LA DISTRIBUTION D'ENERGIE	7
2.2	DISTRIBUTION PAR "BUS BAR" TECHNOLOGIE D'AVENIR ?	7
3	INTERETS DIDACTIQUES	10
4	LES INTERFACES ELECTRONIQUES	11
4.1	CONNEXION AU NŒUD	11
4.2	LIAISON DE DONNEES	13
5	VEHICULE MULTIPLEXE DIDACTIQUE VMD01	19
5.1	PRESENTATION	19
5.2	STRUCTURES ELECTRONIQUES DES DIFFERENTS MODULES	20
5.3	DEVELOPPEMENTS LOGICIELS	25
6	LIENS INTERNETS "SYMPA" ET UTILES	34

1 GENERALITES

Historique

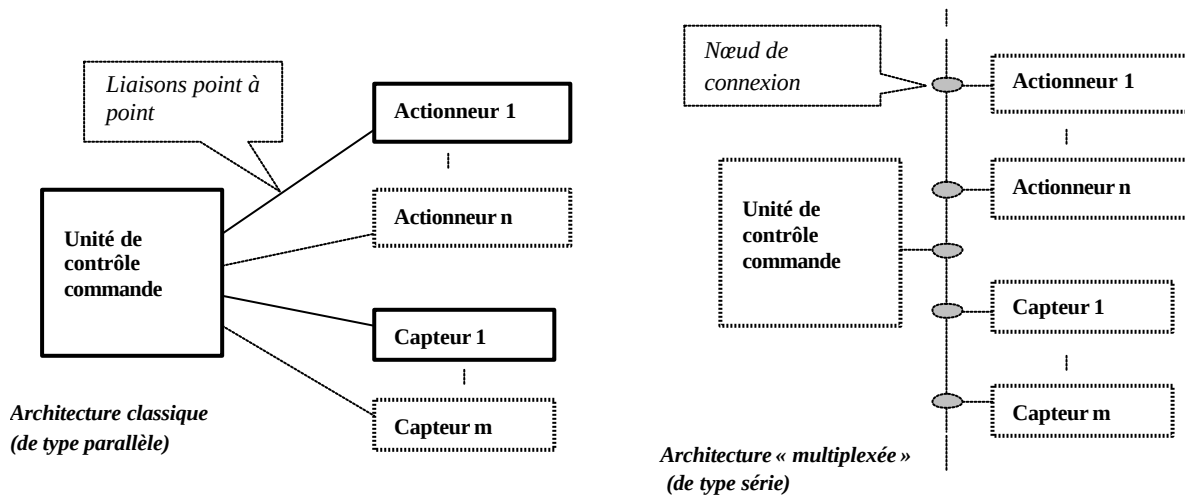
Le bus CAN (Controller Area Network) a initialement été développé par la compagnie allemande Robert Bosch GMBH au milieu des années 80, avec comme objectif de fournir un bus de communication puissant et à un coût réduit pour l'électronique embarquée dans les automobiles. Si l'on se réfère aux graphiques ci-dessous, il était effectivement nécessaire de proposer une alternative à l'augmentation constante de la quantité de câbles dans les véhicules.



Principe du « multiplexage »

La technique dite du multiplexage (terme employé dans l'automobile) permet de transmettre sous forme série, des informations de type électronique ou électrique, sur un seul et même support (fil, câble, barre...).

Sur ce support, viennent se connecter des nœuds de communication, qui reçoivent et émettent les informations. Cet ensemble est appelé réseau. Il est comparable à un réseau informatique de terrain (RLI Réseau local industriel).



Conséquence

La connexion sur le support physique du bus nécessite une interface électrique permettant de satisfaire les contraintes électriques et supportant le protocole de communication.

On trouve sur le marché des capteurs et des actionneurs qui intègrent cette fonction d'interface. Ce sont des capteurs ou actionneurs « intelligents ».

Propriétés

La structure du protocole CAN possède implicitement les propriétés principales suivantes :

- hiérarchisation des messages,
- garantie des temps de latence,
- souplesse de configuration,
- fonctionnement multimaître,
- détections et signalisations d'erreurs,
- retransmission automatique des messages altérés dès que le bus est de nouveau au repos,
- distinction d'erreurs : d'ordre temporaire ou de non-fonctionnalité permanente au niveau d'un nœud,
- déconnexion automatique des nœuds défectueux.

Applications dans l'automobile

La première utilisation du CAN en série dans l'automobile, date de 1992 avec une classe S (Mercedes). Bien entendu, il reste encore plusieurs réseaux en fonction des constructeurs automobiles, mais une tendance se dégage vers le bus CAN. Pour mémoire, GM et FORD utilisent le J1850 avec deux protocoles différents, en Allemagne, c'est exclusivement le bus CAN.

En France, PSA a tout d'abord opté pour le bus VAN (très proche du bus CAN au niveau du protocole) mais utilise aujourd'hui le réseau CAN sur les modèles récents (607, 307 et même 206 depuis juillet 2001).

Dans un même véhicule, plusieurs réseaux "multiplexés" vont cohabiter :

- **un réseau carrosserie** (optiques, rétroviseurs, vitres, ...) (faible débit : 32,5 à 62,5 kB/s, peu d'informations, composants faible coût, temps de réponse demandé 100ms) ;
- **un réseau intersystèmes** (ASR, ABS, ...) (débit moyen : 125 à 500 kB/s, peu d'informations, temps de réponse demandé 10ms) ;
- **un réseau confort** (auto-radio, climatisation, système de navigation...) (débit moyen : 125 à 1MB/s, grande quantité d'informations, temps de réponse faible 500ms) ;
- **un réseau diagnostic (appelé aussi liaison K)** (débit faible : 10kB/s, grande quantité d'informations, temps de réponse de l'ordre de la seconde).

Autres applications

Du fait de son efficacité et de sa robustesse éprouvée, le bus CAN est également de plus en plus utilisé dans nombre d'applications industrielles (on peut citer par exemple : le contrôle des systèmes de navigation, les ascenseurs, les machines agricoles, les photocopieurs, les systèmes médicaux, l'industrie textile, les machines outils, ...).

Normalisation

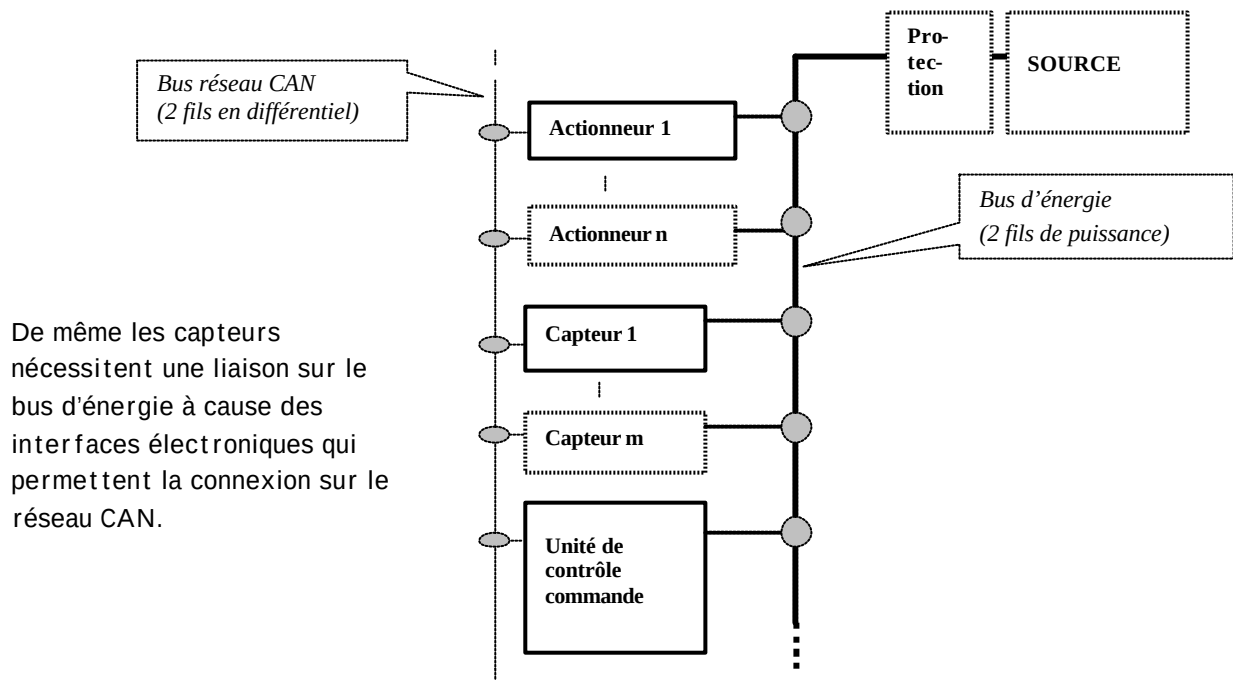
La communication par réseau CAN est un standard international, défini dans la norme **ISO 11898** (pour des applications haut débit) et dans la norme **ISO 11519** (pour des applications faibles et moyens débits).

2 LA DISTRIBUTION D'ENERGIE

2.1 Influence du « multiplexage » sur la distribution d'énergie

Le « multiplexage » s'accompagne d'une remise en cause du principe de distribution de l'énergie ainsi que des protections. Les actionneurs sont connectés en parallèle sur un bus d'énergie.

De même les capteurs nécessitent une liaison sur le bus d'énergie à cause de l'interface électrique permettant la connexion sur le réseau CAN.



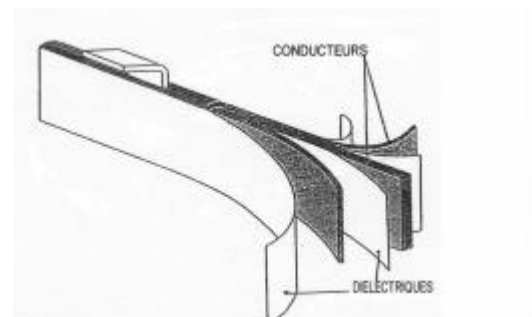
De même les capteurs nécessitent une liaison sur le bus d'énergie à cause des interfaces électroniques qui permettent la connexion sur le réseau CAN.

2.2 Distribution par "bus bar" technologie d'avenir ?

Qu'est-ce qu'un "bus bar" ?

Un bus bar est une barre composée de quatre couches de cuivre superposées l'une sur l'autre (conducteurs), entre lesquelles se trouvent des couches d'isolant (diélectriques).

Parmi les quatre conducteurs, deux fournissent la puissance et les deux autres transmettent la commande (CAN).



Le comparatif suivant permet de mieux apprécier les avantages du bus bar par rapport à un faisceau de fils traditionnels :

Faisceau traditionnel	Bus bar
Montage long et difficile,	Montage simple et rapide, pouvant même être automatisé,
Isolant épais,	Isolant très fin à haute performance diélectrique résistant à de hautes températures (180°C),
N'a pas de tenue mécanique, difficile à stocker et à manipuler.	Structure compacte et rigide, simple à manipuler et à stocker.

De plus, le bus bar présente des caractéristiques électriques intéressantes :

- distribution de puissance élevée (>3000W) avec une très faible chute de tension,
- constitution d'une très faible impédance,
- par rapport à un faisceau classique, réduction des bruits interférants,
- bonne dissipation thermique.

Exemple de réalisation

Mise en situation :

Dans le cadre d'un projet industriel, proposé par une société de sous-traitance spécialisée dans l'étude et la fabrication de connecteurs et de bus barre. Cette étude a été confiée à l'UTBM (Université de Technologie de Belfort Montbéliard)

Eléments du cahier des charges :

Etudier et réaliser une maquette de démonstration permettant de montrer aux constructeurs automobiles la faisabilité d'une distribution de puissance et de signaux de contrôle-commande par bus barre.

Contraintes essentielles :

Cette maquette devait être évolutive et comporter les différents organes électriques d'une automobile :

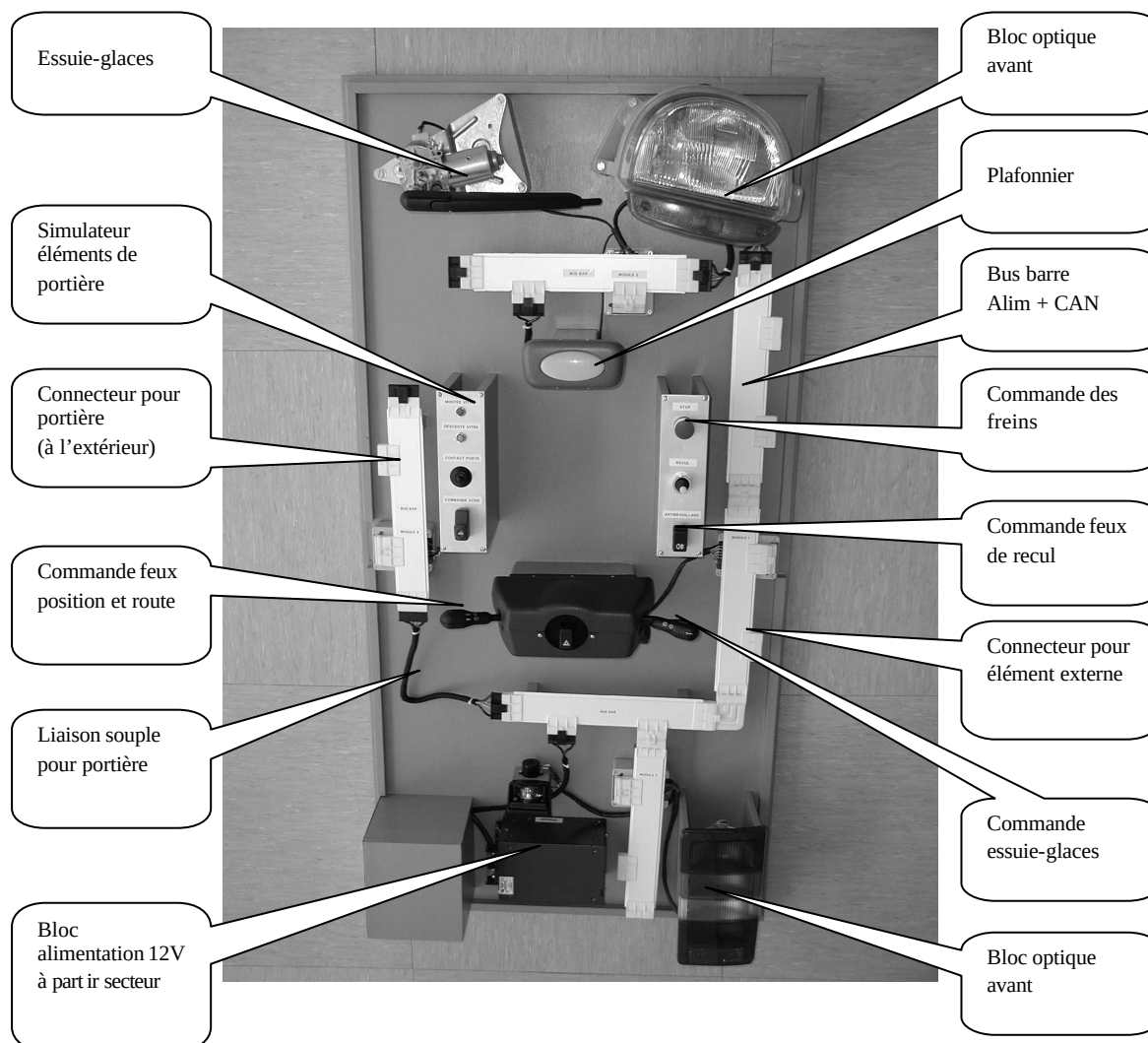
- bloc optique avant
- bloc optique arrière
- commande de feux avant et arrières
- commande de feux de détresse
- commande des essuie-glaces
- moteur d'essuie-glaces
- commande de lève-vitres
- moteur de lève-vitres
- plafonnier.

Elle devait permettre la connexion d'éléments supplémentaires externes (une portière par exemple)

L'alimentation devait pouvoir se faire soit à partir d'une batterie 12V, soit à partir du secteur 220V.

Les modules électroniques réalisant l'interface de connexion au bus barre devait être de taille réduite et peu visible et les liaisons mécaniques facilement démontables.

Vue de dessus de la maquette réalisée :



3 INTERETS DIDACTIQUES

Une séquence d'apprentissage élaborée autour de travaux pratiques sur un système didactique mettant en œuvre un réseau CAN est très riche d'enseignements :

→ sur la connaissance des concepts généraux utilisés dans les réseaux locaux industriels (RLI), notamment le modèle O.S.I. (**O**pen **S**ystems **I**nterconnection),

→ sur les normes en vigueur,

→ sur les termes usuels (voire normalisés) employés,

→ sur la caractérisation d'une communication par RLI,

→ sur les interfaces électriques et électroniques nécessaires,

→ sur les protocoles de communication employés,

→ sur la configuration des éléments configurables,

→ sur la structuration des programmes les mettant en œuvre.

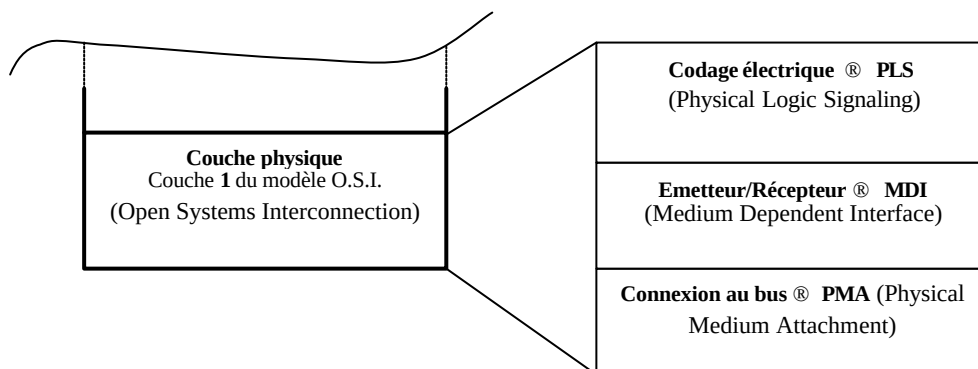
4 LES INTERFACES ELECTRONIQUES

4.1 Connexion au nœud

Intérêt didactique

Etre capable d'analyser les fonctions électroniques permettant la connexion d'un organe électrique sur un réseau local :

- ® satisfaire les caractéristiques électriques impératives permettant une connexion au médium, ce qui correspond à la couche 1 du modèle OSI,
- ® connaître les connecteurs usuellement utilisés (voire normalisés et les brochages),



Développement

Le support de transmission (appelé « médium »)

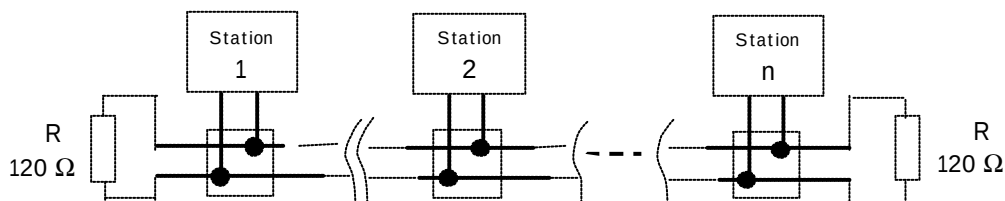
Les signaux électriques circulent sur une liaison à deux fils en mode différentiel (appelée « paire différentielle »).

La longueur du bus

La longueur maximale du bus dépend de la charge capacitive induite par le médium et de la vitesse de transmission souhaitée : pour une longueur inférieure à 30m la vitesse peut atteindre 1 Mbits /S.

Les résistances de terminaison

Les lignes de bus doivent se terminer par des résistances dites de terminaison de valeur 120 Ω placées à chacun des deux bouts



Les caractéristiques électriques au niveau du bus

C'est la différence de potentiel entre les deux fils de transmission constituant le médium (repérés CANH et CANL) qui impose le niveau logique :

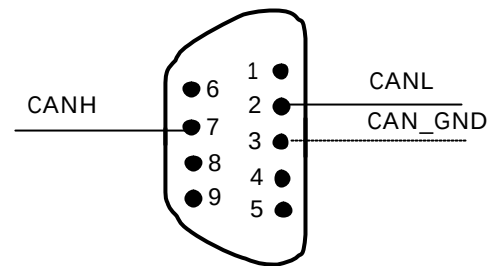
$$\begin{aligned} \text{VCANH} > \text{VCANL} + 0,5\text{V} &\rightarrow 1 \text{ logique} \\ \text{VCANH} < \text{VCANL} - 0,5\text{V} &\rightarrow 0 \text{ logique} \end{aligned}$$

Remarques :

- De part la nature différentielle de la transmission du signal sur le bus, l'immunité électromagnétique (EMI) est assurée car les deux lignes sont toutes deux affectées, de la même manière, par un rayonnement perturbateur.
- Au repos (en l'absence de signal transmis), les niveaux de potentiels sont :
VCANH = 2,5V et VCANL = 1,5V soit un niveau logique 0

Le connecteur et le brochage :

Dans les installations industriels, le connecteur préconisé est le « DB9 » mâle pour le connecteur sur le module et femelle pour le connecteur sur le médium, avec le brochage donné sur la figure ci-contre.

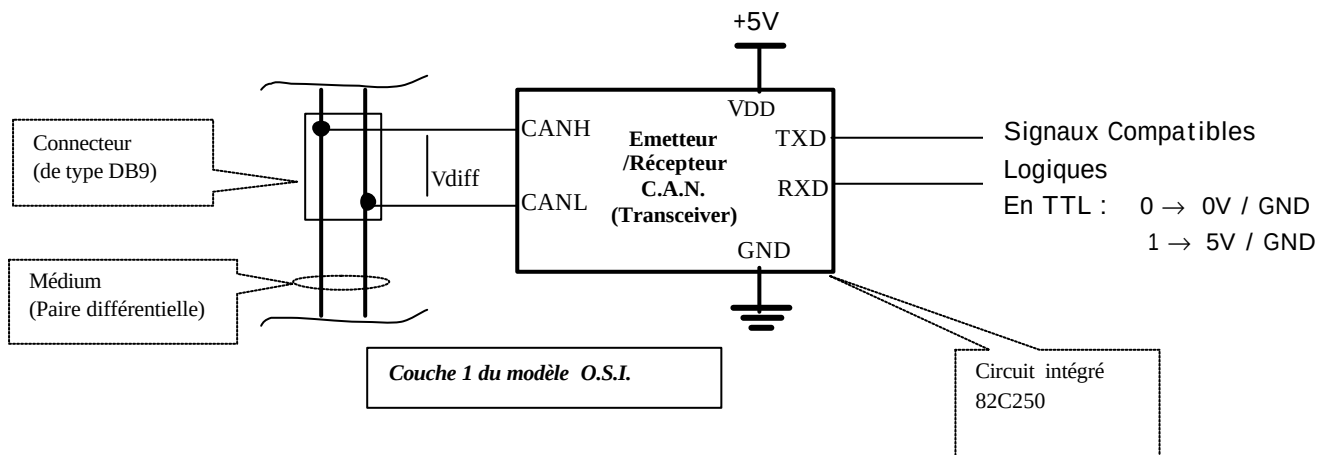


Les circuits intégrés réalisant l'interface électrique ("Transceiver C.A.N.")

On trouve sur le marché des circuits intégrés réalisant l'interface entre les signaux électriques au niveau du médium et les signaux acceptables par le composant électronique réalisant la couche 2 du modèle OSI :

- composant 80C250 du constructeur PHILIPS Semi-conductors
- composant MCP2551 du constructeur MICROCHIP

Soit le schéma de principe d'un nœud réalisant la couche 1 du modèle O.S.I.



Remarque

En émission le signal arrive en TXD, donc se retrouve sur le bus, puis sur la sortie RXD. Une comparaison du signal reçu en échos avec le signal émis est la base du principe d'arbitration lorsque plusieurs stations prennent l'initiative d'émettre en même temps (voir chapitre suivant). Cette propriété permet également, en cours d'émission, de détecter des différences entre bits émis et bits reçus.

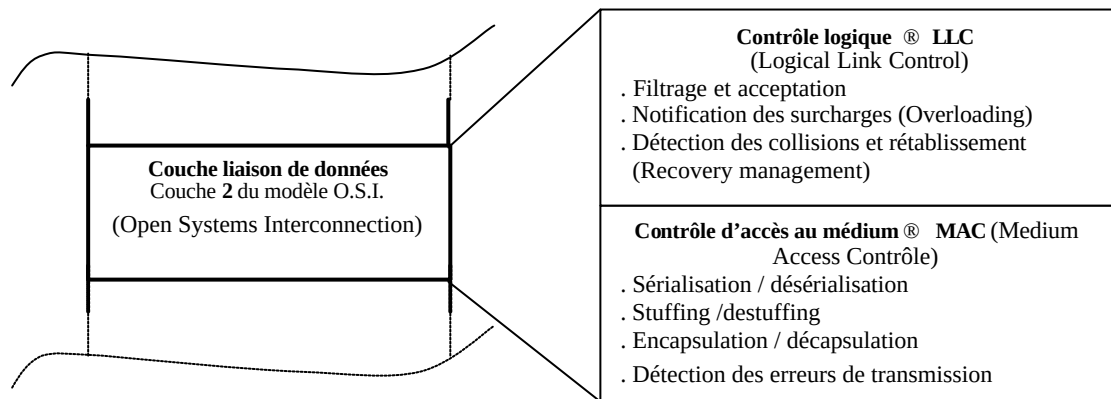
4.2 Liaison de données

Intérêt didactique :

Etre capable d'analyser et de configurer les fonctions électroniques permettant d'établir une communication via un réseau local industriel pour :

- ® satisfaire les caractéristiques temporelles imposées (respect de la vitesse de transmission)
- ® respecter la constitution des trames de données,
- ® permettre la sécurisation du lien (filtrage, acceptation, vérification de la taille de la trame)
- ® connaître le principe d'arbitration.

Ces caractéristiques et techniques correspondent à la couche 2 du modèle OSI.



Développement

Codage NRZ → Non Retour à Zéro

Pendant toute la durée de la transmission d'un bit le niveau de tension est maintenu constant.

Vitesse de transmission

La vitesse de transmission, appelée "Baud Rate" est le nombre de bits transmis par seconde (l'unité est le bit/S ou le "baud").

La durée pendant laquelle un bit est transmis est égale à l'inverse de la vitesse de transmission.

Une communication à 1 méga bit/S = 1 méga baud = 10^6 bit/s entraîne une durée de transmission d'un bit égale 10^{-6} S = 1µS .

« Stuffing » et « Destuffing »

Au cours de la transmission d'une trame, lorsqu'un grand nombre de bits au même état logique se succèdent, la ligne reste dans un même état trop longtemps. Pour éviter cet état de fait, la technique du « bit stuffing » impose au transmetteur d'ajouter automatiquement un bit d'état opposé lorsque 5 bits successifs ont même état. Naturellement l'opération inverse doit être effectuée en réception, c'est le « destuffing »

ARBITRAGE

Le bus CAN est de type "multi-maître" c'est à dire qu'à un instant donné, à partir d'un état où le bus est libre, plusieurs émetteurs potentiels peuvent vouloir prendre la main pour transmettre un message.

Le procédé d'attribution du bus est basé sur le principe de "l'arbitrage bit à bit" selon lequel les nœuds en compétition, émettant simultanément sur le bus, comparent bit à bit l'identificateur de leur message avec celui des messages concurrents.

Les stations sont connectées sur le bus par des sorties dites "à collecteur ouvert".

Le zéro logique est donc imposé par un transistor à l'état passant (basse impédance) alors qu'un 1 logique est imposé par un transistor bloqué (haute impédance). En cas de conflit l'état 0 écrase donc l'état 1.

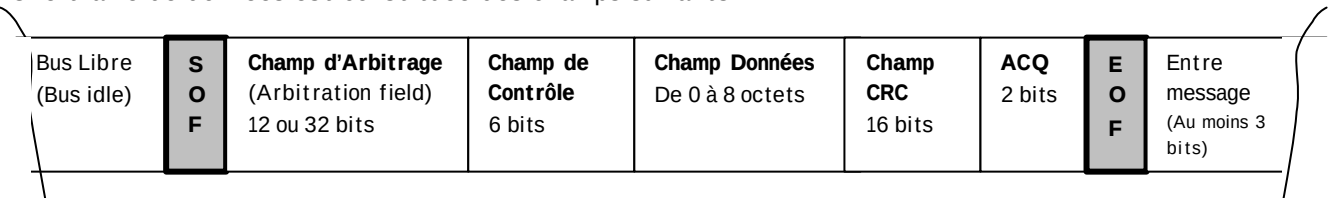
L'état 0 est donc l'état "dominant" alors que l'état 1 est donc l'état "récessif".

Lors de l'arbitration bit à bit, dès qu'une station émettrice se trouve en état récessif et détecte un état dominant, elle perd la compétition et arrête d'émettre.

Tous les perdants deviennent automatiquement des récepteurs (listening only – à l'écoute uniquement) et ne tentent à nouveau d'émettre que lorsque le bus se libère.

Format d'une trame de données (DATA FRAME)

Une trame de données est constituée des champs suivants :



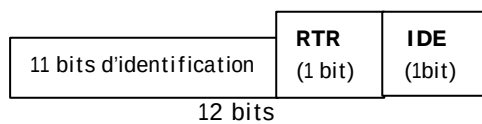
SOF Start Of Frame (Début de trame) (sur 1 bit)

D'une durée d'un bit, son état est mis à 0 (état dominant) par la station qui prend l'initiative de débiter une transmission alors que le bus est libre.

Champ d'Arbitrage (sur 12 ou 32 bits)

Le champ d'arbitrage est composé de l'identificateur et d'un bit permettant d'informer du type de trame (bit **RTR** Remote Transmission Request)

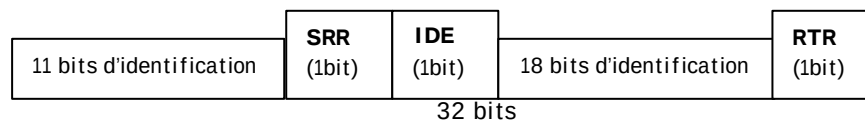
Format « standard » (version 2.0A du bus CAN):



RTR = 1 pour une trame de donnée (DATA frame)

RTR = 0 pour une trame d'interrogation (Remote frame)

Format « étendu » (Version 2.0B du bus CAN):



- Le bit **IDE** informe de la taille de l'identificateur :

→ est à 0 (dominant) en format standard

→ est à 1 (récessif) en format étendu.

Il est dans la zone de contrôle en mode standard et dans la zone d'identification en mode étendu.

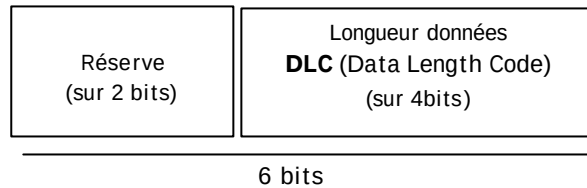
- Le bit **SRR**

(Substitute Remote

Request) se substitue en lieu et place du bit RTR de format standard. Il est mis à 1. Ce bit est utile lorsqu'une trame au format étendu circule sur un bus CAN et rencontre une trame classique qui possède le même identificateur (partie haute).

Champ de Contrôle (sur 6 bits)

Ce champ comprend deux bits de réserve et 4 bits définissant la taille en nombre d'octets des données transmises (ce nombre est entre 0 et 8)

**Champ de données**

Les différentes données utiles transmises. Sa longueur dépend du nombre d'octets transmis défini dans le champ DLC.

Champ CRC (Cyclic Redundancy Code) sur 16 bits

Lors de l'émission, l'émetteur effectue un calcul (normalisé) sur les données à transmettre. Le résultat de ce calcul est transmis dans le champ CRC. Le récepteur effectue le même calcul et compare le résultat avec celui de l'émetteur transmis. Si une erreur est détectée, le récepteur renvoie un message d'erreur sous forme d'une trame interrogative.

Champ d'ACQuitement sur 2 bits (1 bit Slot suivi d'1 bit délimiteur)

L'émetteur envoie un bit à 1 (Récessif) et un récepteur ayant reçu correctement le message impose un zéro (dominant) : il acquitte donc le message.

EOF End Of Trame

Chaque trame est terminée par 7 bits à 1 logique (récessif).

Format d'une trame d'interrogation (Remote frame)

Une station peut demander à une autre station de lui envoyer des informations qui lui sont utiles pour accomplir sa mission. Cette interrogation se réalise par trame spécifique appelée "Remote Frame". Deux particularités distinguent une trame d'interrogation (Remote Fram) d'une trame de donnée classique :

- le bit RTR est placé à 0 (dominant) alors qu'il est à 1 dans une trame de données,
- elle ne contient pas de données.

Trame d'erreur (error frame)

De part le mode de connexion au bus (voir chapitre précédent), toute station qui émet un bit le reçoit également. En dehors de la phase d'arbitrage, elle peut détecter une différence entre le bit transmis et le bit renvoyé en échos.

Si une station détecte une erreur, elle renvoie une trame d'erreur constituée de 6 bits à l'état 0 (dominant) suivi de 8 bits « délimiteurs » à 1 (récessif). Plusieurs stations peuvent détecter l'erreur dans la transmission et envoyer elles aussi une trame d'erreur. Il y aura alors superposition de bits d'erreur (dominant).

6 bits d'erreur (à 0 logique)	De 0 à 6 bits d'erreur à 0 log « superposition »	8 bits « délimiteur » (à 1 logique)	3 Bits inter trame
-------------------------------------	--	---	------------------------------

A l'issue des 8 bits « délimiteurs », le bus est libéré et les communications peuvent reprendre.

Remarques :

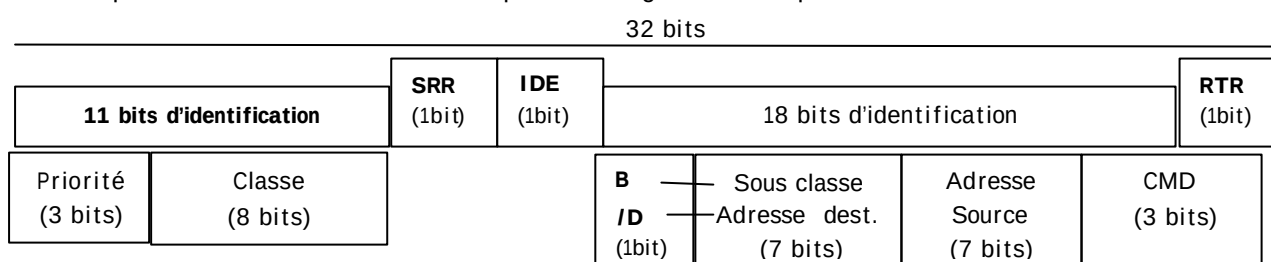
- Le mode de fonctionnement du réseau CAN, avec sa technique de détection des erreurs en font un moyen de communication très sûr (la littérature sur le sujet parle d'1 erreur non détectée pour 1000 années de fonctionnement ou encore d'une probabilité d'erreur résiduelle inférieure à $4,6 \cdot 10^{-11}$!)
- Tout ce système de gestion des erreurs est transparent pour le développeur et l'utilisateur. Les composants intégrés gèrent automatiquement les conflits et les erreurs. Une station en défaut trop fréquemment se déconnecte automatiquement du réseau.

Intérêt du mode étendu

Le fait que l'identificateur comporte un plus grand nombre de bits permet de mieux structurer l'application car il pourra contenir plus d'informations.

Exemple de structuration possible:

Si on reprend le format de la trame on peut envisager les champs d'informations suivants:



→ La **priorité** permet de donner plus ou moins d'importance au message. Il faut se rappeler que sur le bus l'état 0 est dominant et que l'état 1 est récessif, ce qui entraîne qu'un niveau 0 est prioritaire sur un niveau 1.

La priorité étant codée sur 3 bits, son niveau (sa valeur) sera comprise entre 0 et 7 avec le niveau 0 le plus prioritaire.

→ La **classe** permet de caractériser le type d'information qui circule sur le bus. Le codage sur 8 bits de la classe permet d'envisager une classification en 256 types. Suivant le même principe de dominance du 0 sur le 1, pour deux messages de même niveau de priorité, c'est celui dont la classe est la plus faible qui sera dominant (qui l'emportera).

→ Le bit **B/D** (**B**roadcast / **D**irected).

Si ce bit est positionné à 1, le message sera général c'est à dire qu'il n'a pas de destinataire particulier. Il est destiné à tous les nœuds. Dans ce cas, les 8 bits suivants seront destinés à la définition d'une sous-classe (128 sous-classes possibles et plus la sous-classe à un poids faible, plus elle est prioritaire).

Si ce bit est positionné à 0, c'est que le message a un destinataire désigné. Dans ce cas, les 7 bits suivants donnent l'adresse du destinataire (un destinataire désigné parmi 128).

Suivant le même principe de dominance du 0 sur le 1, pour deux messages identiques jusqu'à là, c'est le message qui possédera un destinataire qui sera prioritaire.

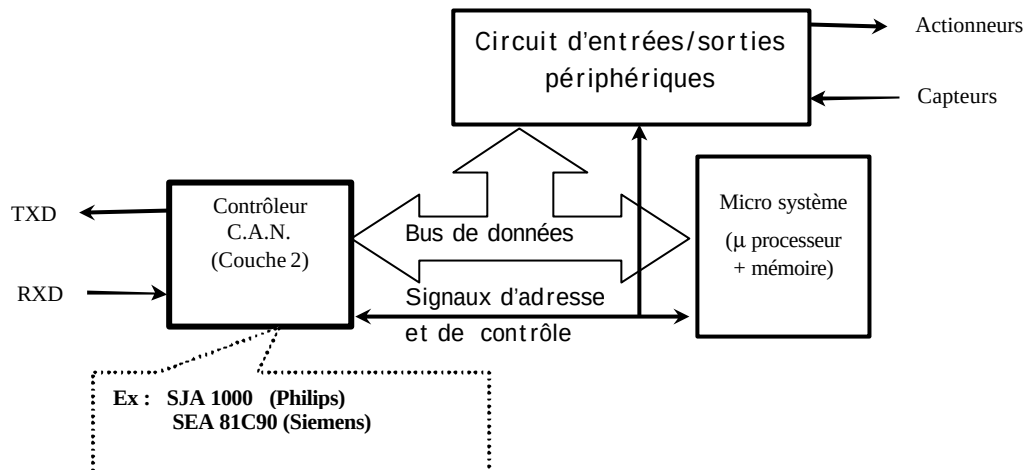
→ L'**adresse source** désigne l'émetteur du message. Son codage sur 7 bits (comme c'est le cas pour le destinataire) indique que le réseau CAN pourra avoir 128 nœuds.

→ La **CoMmanDe** définit sur 3 bits une information de commande, comme une action particulière sur un nœud particulier.

Les composants intégrés matérialisant la couche 2 du modèle OSI

On distingue plusieurs familles de circuits intégrés, suivant le type de système auquel ils seront reliés :

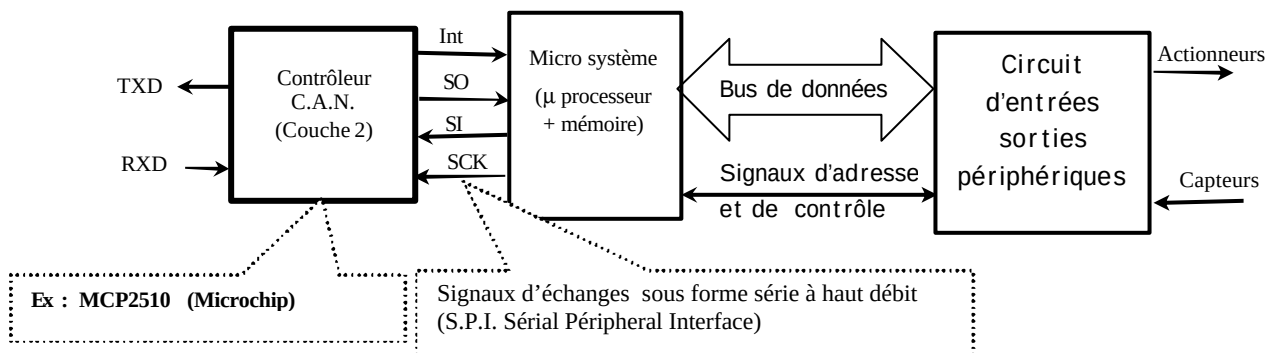
→ le contrôleur est relié en mode parallèle à un système à base de microprocesseur (ou microcontrôleur) qui lui même réalise l'interface avec les actionneurs et capteurs.



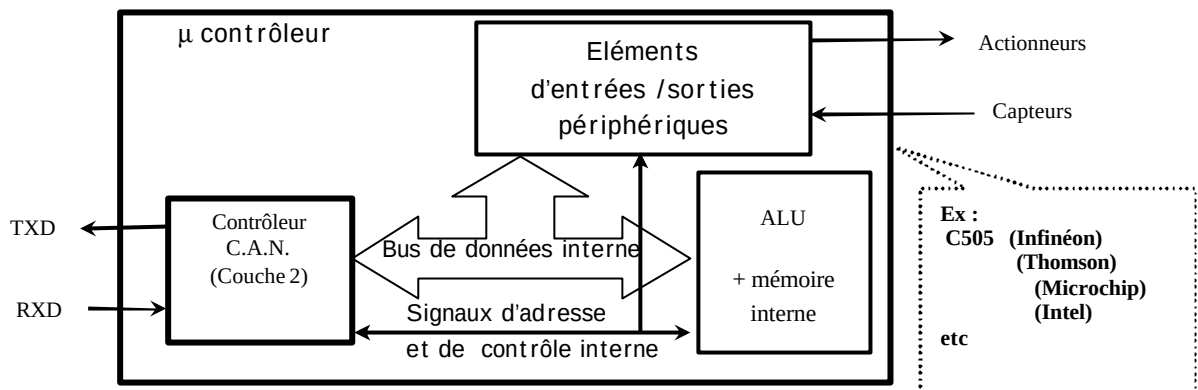
Remarque:

- Parmi les signaux de contrôle il y a un signal d'interruption émis par le contrôleur CAN qui permet donc de gérer les échanges en interruption.
- C'est cette solution qui a été mise en œuvre pour réaliser l'ordinateur de bord, contenant le programme de gestion de l'application (nœud maître).

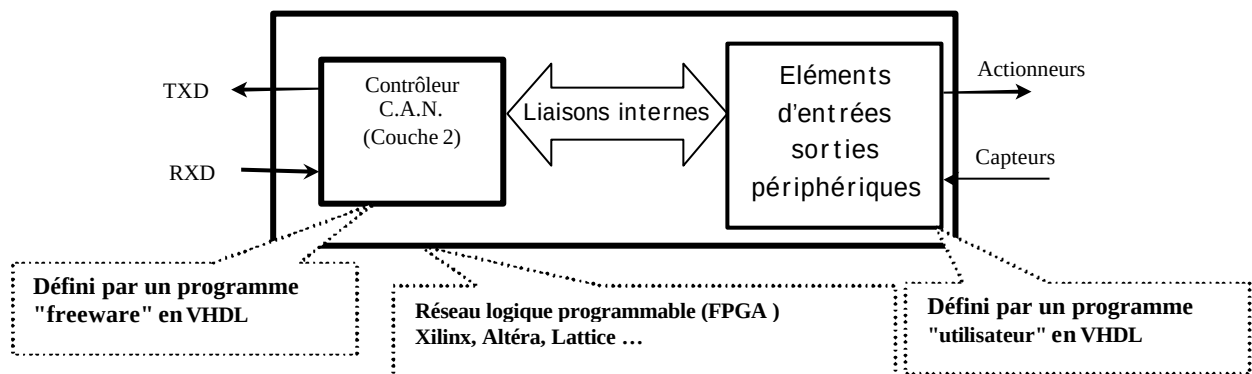
→ le contrôleur est relié en mode série à haut débit à un système à base de microprocesseur (ou microcontrôleur) qui lui même réalise l'interface avec les actionneurs et capteur.



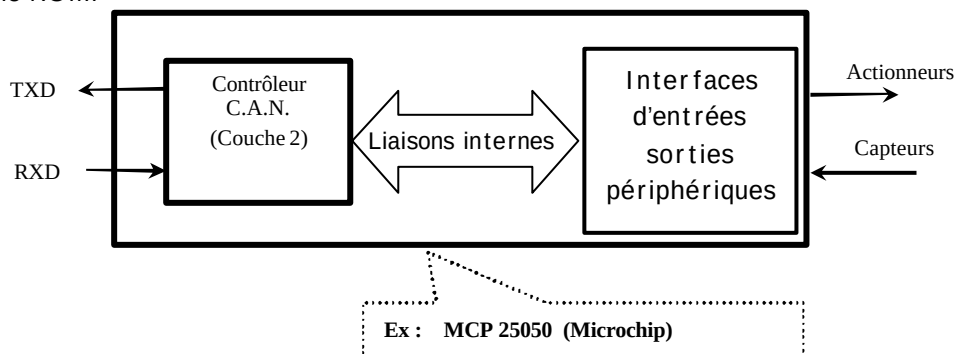
→ le contrôleur CAN est intégré dans le microcontrôleur qui lui même intègre l'interface avec les actionneurs et capteurs.



→ Le contrôleur est réalisé par un réseau logique programmable (FPGA). Le programme de la fonction est fourni sous la forme d'un programme source écrit en VHDL (**V**ery high speed **H**ardware **D**escription **L**angage). L'utilisateur rajoute les fonctionnalités utiles à son application sous la forme d'un programme en VHDL également.



→ le contrôleur CAN intègre l'interface avec les actionneurs et capteurs. Un outil de configuration permet de personnaliser l'application. Cette configuration est gravée dans le circuit dans une zone ROM.

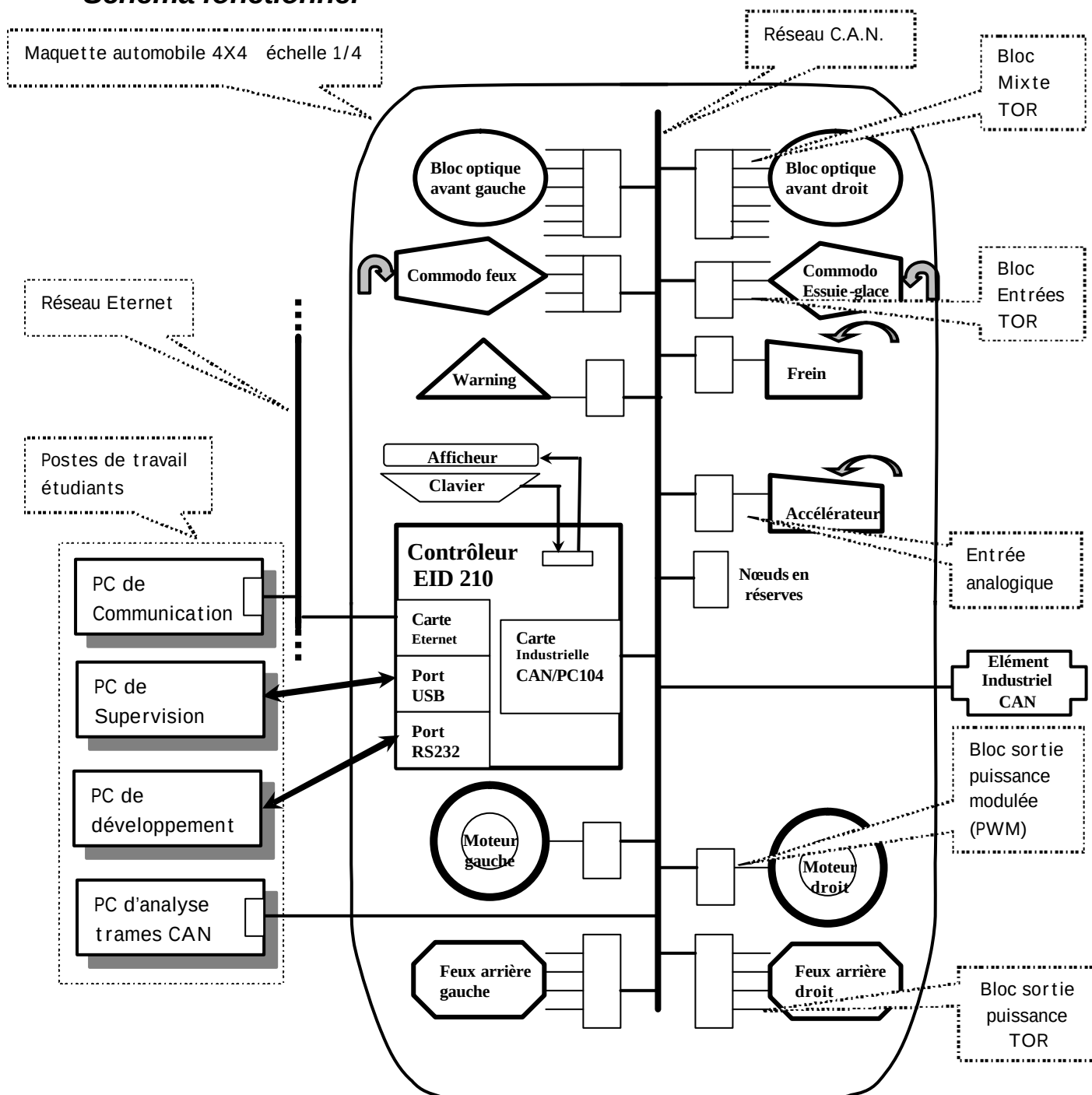


C'est cette dernière solution qui a été choisie pour interfacer les différents éléments de la maquette (Blocs optiques, commodos, moteurs, capteurs etc ...)

5 VEHICULE MULTIPLEXE DIDACTIQUE VMD01

5.1 Présentation

Schéma fonctionnel



5.2 Structures électroniques des différents modules

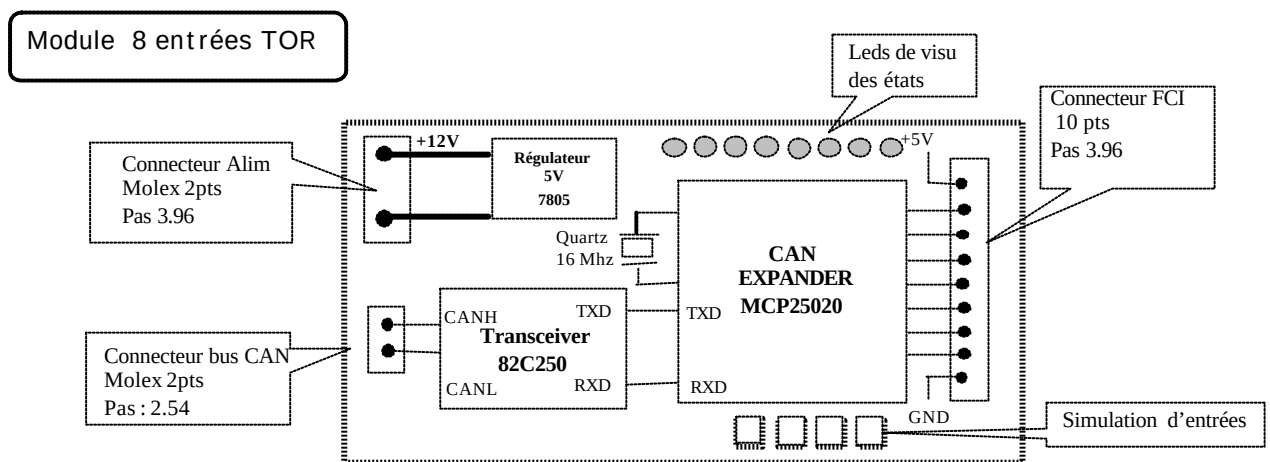
Intérêt didactique :

Etre capable d'analyser une fonction électronique en vue de son utilisation afin de satisfaire un cahier des charges imposé.

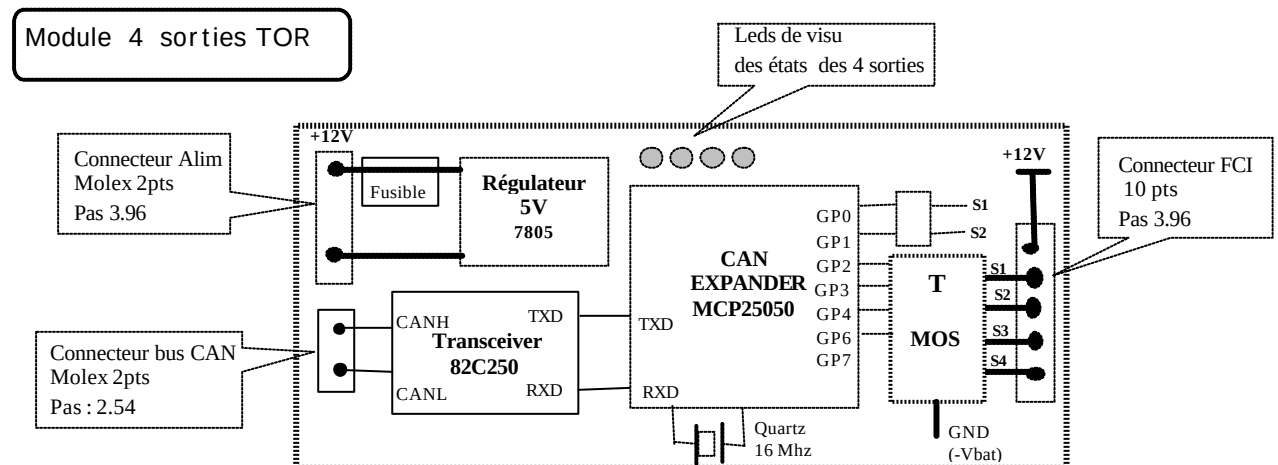
Etre capable de définir et d'écrire les fonctions logicielles pour accéder correctement aux diverses fonctionnalités du module utilisé.

Etre capable de lire, comprendre et parfois interpréter la notice d'un composant programmable.

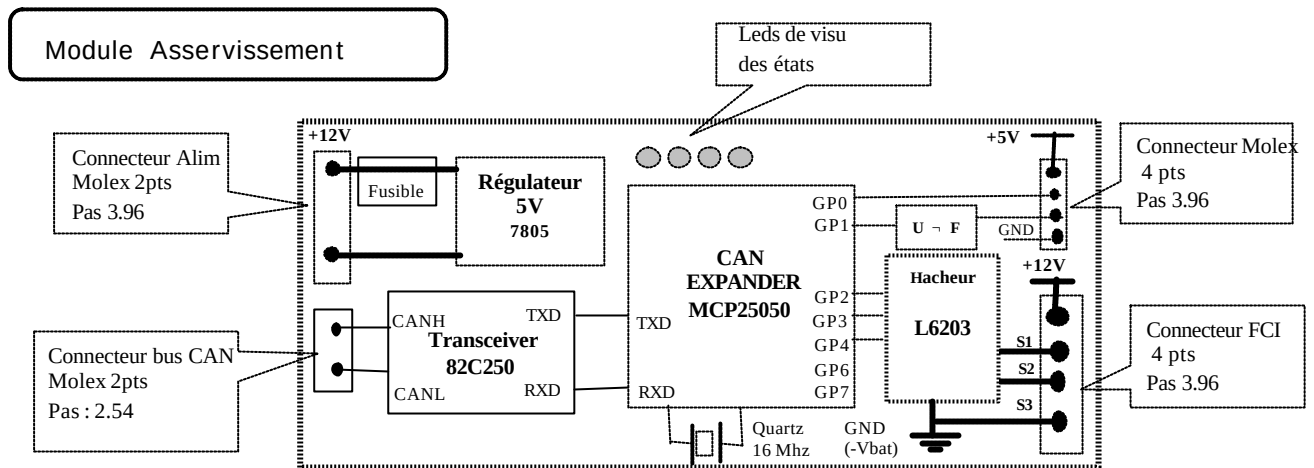
Les différents modules installés sur le système didactique



Dans la mesure ou l'état est visualisé par des « leds », il est possible d'utiliser ce module comme simulateur de 7 sorties (l'entrée GP7 ne peut jamais être configurée en sortie)



Les sorties de puissance S1 et S2 sont contrôlées c'est à dire qu'il sera possible de mesurer le courant dans la charge. Les deux entrées GP0 et GP1 sont configurées en entrées analogiques. Cette fonction permet par exemple de savoir si une ampoule de phare est grillée.



Dans ce module, on dispose d'une sortie modulée (PWM) en bipolaire. La tension moyenne de sortie peut donc varier de -12V à +12V.

La sortie GP2 sera donc configurée en sortie « PWM ». Elle définira donc le module de la valeur moyenne. La sortie GP3 sera configurée en sortie logique et définira la polarité. La sortie GP4 valide ou inhibe la puissance.

L'entrée GP0 sera configurée en entrée analogique et permettra donc d'acquérir une information de mesure sous forme analogique.

L'entrée GP1 sera elle aussi configurée en entrée analogique et permettra donc, à travers un convertisseur fréquence → tension d'acquérir une information de mesure d'un signal logique à une certaine fréquence (capteur de vitesse optique).

Ce module permettra donc de réaliser un asservissement de vitesse ou une régulation de température.

Présentation succincte du circuit d'interface MCP2502X/5X

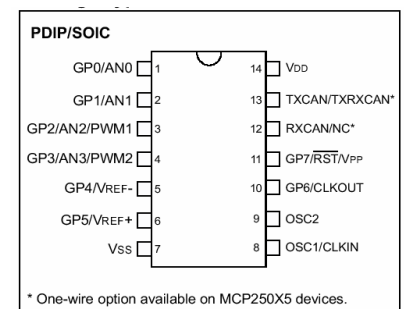
C'est un circuit intégré 14 broches disponible en version PDIP ou SOIC.

Côté interface CAN il satisfait la norme CAN V2.0B c'est à dire qu'il peut communiquer avec une vitesse de transmission qui peut atteindre 1Mbit/s. Certaines versions du circuit permettent une communication sur 1 fil (One-wire).

Côté application, il possède 8 lignes d'entrées/sorties (GP0 à GP7) configurables individuellement en entrée ou en sortie. Seule la ligne GP7 ne peut être utilisée en sortie.

Si on le souhaite, il est capable d'envoyer un message sans qu'il soit interrogé si l'une de ses entrées change d'état.

Deux liaisons (GP2 et GP3) peuvent être configurées en sorties modulées (PWM). Ces deux sorties peuvent être commandées indépendamment l'une de l'autre, fréquences et rapports cycliques sur 10 bits.



Device	A/D	One-wire CAN
MCP25020	No	No
MCP25025	No	Yes
MCP25050	Yes	No
MCP25055	Yes	Yes

Certaines versions du circuit intègrent un convertisseur analogique → numérique (4 voies) sur 10 bits. Si on le souhaite, il est capable d'envoyer un message sans qu'il soit interrogé si l'une de ses entrées analogique dépasse des seuils de tension que l'on peut choisir.

Il possède un "scheduler" qui lui permet d'envoyer un message à intervalles de temps réguliers sans qu'on le lui demande (par exemple l'état de ses entrées ou la valeur convertie d'une des entrées analogiques).

C'est un circuit configurable grâce à une banque de registres qui sont gravés dans le circuit lors d'une phase de programmation.

Remarque :

- Ces circuits ne sont pas reprogrammables.
- Le constructeur (MICROCHIP) commercialise des outils logiciels et matériel de configuration et de programmation.
- Un circuit déjà programmé peut être lu par le logiciel de configuration et de programmation.

La configuration de la vitesse de transmission

Cette vitesse dépend de la fréquence du signal d'horloge interne (imposée par le quartz) notée t_{osc} . Cette fréquence est divisée (passage dans un « prescaler ») pour obtenir la période t_Q (Time Quantum).

Le coefficient de division BRP est choisi grâce à un mot de 6 bits BRP5 ... BRP0 (Ces bits font partie du registre de configuration n°1 CNF1 du circuit).

On obtient la valeur de t_Q grâce à l'expression :

$$t_Q = 2 * t_{osc} * (BRP + 1)$$

La durée de transmission d'un bit (Bit Time) est fonction de 3 paramètres :

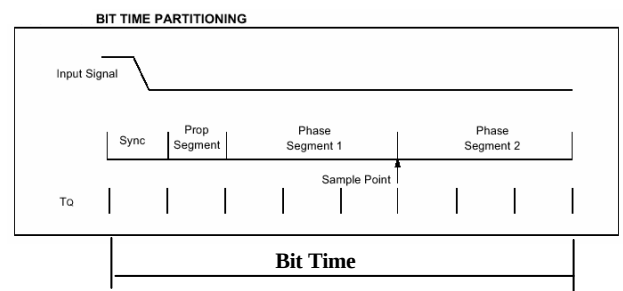
- un paramètre de synchronisation « SYN_Seg »
→ sur deux bits (SJW1 et SJW0) inclus dans le registre de configuration n°1
- un paramètre de propagation « Prop Segment »
→ sur 3 bits PRSEG2, PRSEG1 et PRSEG0 dans le registre de configuration n° 2
- un premier paramètre de phase « Phase Segment 1 »
→ sur 3 bits PHSEG12, PHSEG11 et PHSEG10 dans le registre de configuration n° 2
- un deuxième paramètre de phase « Phase Segment »
→ sur 3 bits PHSEG22, PHSEG21 et PHSEG20 dans le registre de configuration n° 2

Les paramètres de phase permettent de définir l'instant d'échantillonnage (« Sample time»), instant où l'on prend la valeur logique du bus ce qui va donner l'état du bit.

On obtient la valeur de t_{BIT} (durée de transmission d'un bit) grâce à l'expression :

$$t_{BIT} = t_Q * (S_Seg + P_Seg + PH_Seg1 + PH_Seg2)$$

La vitesse de transmission est l'inverse de t_{BIT}



La répartition des paramètres dans les registres est donnée ci-contre.

(Voir « DATA SHEET » du circuit MCP25050 page 8 et 9)

Exemple de calcul de la vitesse de transmission:

La fréquence du quartz sur la carte est 16 Mhz

On a choisi BRP = 4

$$\rightarrow t_Q = 2 \cdot (4+1) \cdot t_{osc} = 10/16\mu s = 0.625 \mu s$$

On a choisi :

SJW = 0, PRSEG = 0

PHSEG1 = 8 et PHSEG2 = 8

D'après « DATA SHEET »

$$\rightarrow t_{BIT} = 16 \cdot t_Q$$

En définitive :

$$t_{bit} = 16 \cdot t_{scl} = 16 \cdot 0.625 \mu s = 10 \mu s$$

Soit une vitesse de $1/t_{bit} = 100$ Kbit/s

CNF1 - CAN CONFIGURATION REGISTER 1

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SJW1	SJW0	BRP5	BRP4	BRP3	BRP2	BRP1	BRP0
bit7							bit0

CNF2 - CAN CONFIGURATION REGISTER 2

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
BTLMODE	SAM	PHSEG12	PHSEG11	PHSEG10	PRSEG2	PRSEG1	PRSEG0
bit7							bit0

CNF3 - CAN CONFIGURATION REGISTER 3

U-0	R/W-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
---	WAKFIL	---	---	---	PHSEG22	PHSEG21	PHSEG20
bit7							bit0

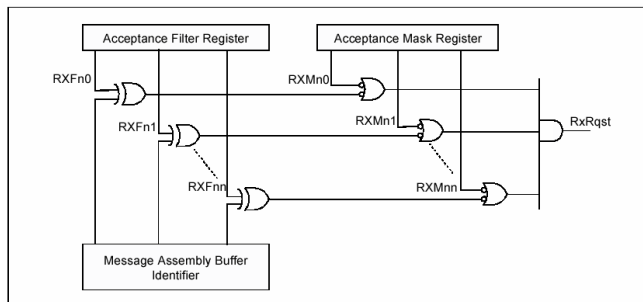
La configuration des masques et des filtres "d'acceptance"

Un message circulant sur le bus ne sera pris en considération par le circuit que si l'identificateur associé au message a passé avec succès les obstacles du filtre et du masque.



Aux habitués du SJA1000, dans ce cas la technique de filtrage et du masquage est différente!

Cette technique peut être déduite du schéma logique donné ci-après, ainsi que de la table de vérité.



FILTER/MASK TRUTH TABLE

Mask Bit n	Filter Bit n	Message Identifier bit n001	Accept or reject bit n
0	X	X	Accept
1	0	0	Accept
1	0	1	Reject
1	1	0	Reject
1	1	1	Accept

Note: X = don't care

En résumé, si on donne à un bit de masque l'état 1 logique un message ne sera accepté que si le bit correspondant de l'identificateur associé est du même état que celui donné au filtre.

Par contre, si on donne à un bit de masque l'état 0 logique, le bit correspondant de l'identificateur est masqué c'est à dire que son état n'est pas vérifié.

La définition des masques et des filtres s'effectue grâce à un certain nombre de registres destiné à cet effet (voir tableau ci-après).

Rappel :

DANS LA VERSION "STANDARD" DU BUS CAN (VERSION V2.0A) L'IDENTIFICATEUR EST SUR 11 BITS. DANS CE CAS LES BITS DE L'IDENTIFICATEUR SONT REPERES **SID10 ... SID0**.

DANS LA VERSION "ETENDUE" DU BUS CAN (VERSION V2.0B) L'IDENTIFICATEUR EST SUR 29 BITS. DANS CE CAS LES BITS DE L'IDENTIFICATEUR SONT REPERES **EID17 ... EID0** (CE SONT LES POIDS FAIBLES DE L'IDENTIFICATEUR, LES POIDS FORTS ETANT L'IDENTIFICATEUR STANDARD.

Il y a deux séries de registres dans lesquels on charge les valeurs des masques et des filtres :

- la série d'indice 0 destinée aux trames interrogatives ("Information Request"),
- la série d'indice 1 destinée aux entrées de données ("Input message").

Les registres définissant les masques															
RXMSIDH - ACCEPTANCE FILTER MASK STANDARD IDENTIFIER HIGH															
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3	SID2	SID1	SID0	SID3*	SID2	SID1	SID0	SID3*
bit7															bit0
R = Readable bit W = Writable bit U = Unimplemented, read as '0'															
RXMSIDL - ACCEPTANCE FILTER MASK STANDARD IDENTIFIER LOW															
R/W-x	R/W-x	R/W-x	U-0	R/W-x	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID2	SID1	SID0	—	EXIDE	—	EID17	EID16	EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit7															bit0
RXMEID8 - ACCEPTANCE FILTER MASK EXTENDED IDENTIFIER MID															
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8	EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit7															bit0
RXMEID0 - ACCEPTANCE FILTER MASK EXTENDED IDENTIFIER LOW															
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0	EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit7															bit0

D'après "DATA SHEET" du circuit pages 12, 13 et 14

Les registres définissant les filtres															
RXFNSIDH - ACCEPTANCE FILTER N STANDARD IDENTIFIER HIGH															
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3	SID2	SID1	SID0	SID3*	SID2	SID1	SID0	SID3*
bit7															bit0
R = Readable bit W = Writable bit U = Unimplemented, read as '0'															
RXFNSIDL - ACCEPTANCE FILTER N STANDARD IDENTIFIER LOW															
R/W-x	R/W-x	R/W-x	U-0	R/W-x	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID2	SID1	SID0	—	EXIDE	—	EID17	EID16	EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit7															bit0
RXFNEID8 - ACCEPTANCE FILTER N EXTENDED IDENTIFIER MID															
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8	EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit7															bit0
RXFNEID0 - ACCEPTANCE FILTER N EXTENDED IDENTIFIER LOW															
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0	EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit7															bit0

Les différents messages

Pour communiquer avec le circuit via le bus CAN, on distingue 3 types de messages :

- messages de demande d'information (Information Request Messages) → IRM qui permettent de demander au circuit de renvoyer la valeur d'un ou plusieurs de ses registres internes (par exemple pour lire l'état de ses entrées),
- messages d'entrée (Input Messages) qui permettent de modifier la valeur d'un ou plusieurs registres internes (par exemple pour modifier l'état de ses sorties),
- messages de sortie (Output messages) qui permettent de répondre à un message d'interrogation.

Interrogations sur la valeur des registres	
Name	Description
Read A/D Registers	Transmits a single message containing the current state of the analog and I/O registers including the configuration
Read Control Registers	Transmits several control registers not included in other messages
Read Configuration Registers	Transmits the contents of many of the configuration registers
Read CAN error states	Transmits the error flag register and the error counts
Read PWM Configuration	Transmits the registers associated with the PWM modules
Read User Registers 1	Transmits a the values in bytes 0 - 7 of the user memory
Read User Registers 2	Transmits a the values in bytes 8 -15 of the user memory
Read Register*	Transmits a single byte containing the value in an addressed user memory register

Pour changer la valeur des registres	
Write Register	Uses a mask to write a value to an addressed register
Write TX Message ID0 (TXID0)	Writes the identifiers to a specified value
Write TX Message ID1 (TXID1)	Writes the identifiers to a specified value
Write TX Message ID2 (TXID2)	Writes the identifiers to a specified value
Write I/O Configuration Registers	Writes specified values to the three IOCON registers
Write RX Mask	Changes the receive mask to the specified value
Write RX Filter0	Changes the specified filter to the specified value
Write RX Filter1	Changes the specified filter to the specified value
*The Read Register command is available when using extended message format only. Not available with standard message format.	

Se reporter à la "DATA SHEET" du circuit (pages 21 et 22) pour connaître la constitution des différents messages.

5.3 Développements logiciels

5.3.1 Le logiciel «Application»

Le logiciel « application » aura pour fonction principale de réaliser le fonctionnel de la machine. Il devra également réaliser les échanges de données avec les autres

Ce logiciel sera exécuté par le processeur MC68332 , CPU du contrôleur EID210.

Intérêt didactique :

Etre capable de définir les structures de données permettant de programmer une application. Celles-ci devra tenir compte des éléments matériels constituant le système.

Etre capable d'analyser un cahier des charges imposé incluant des contraintes temps réel.

Etre capable de définir et de mettre en œuvre une programmation structurée et commentée, en assembleur et en langage évolué « C ».

Etre capable de comprendre et d'utiliser un noyau multi tâche temps réel.

Etre capable de rédiger une notice claire et exploitable pour une maintenance logicielle ultérieure.

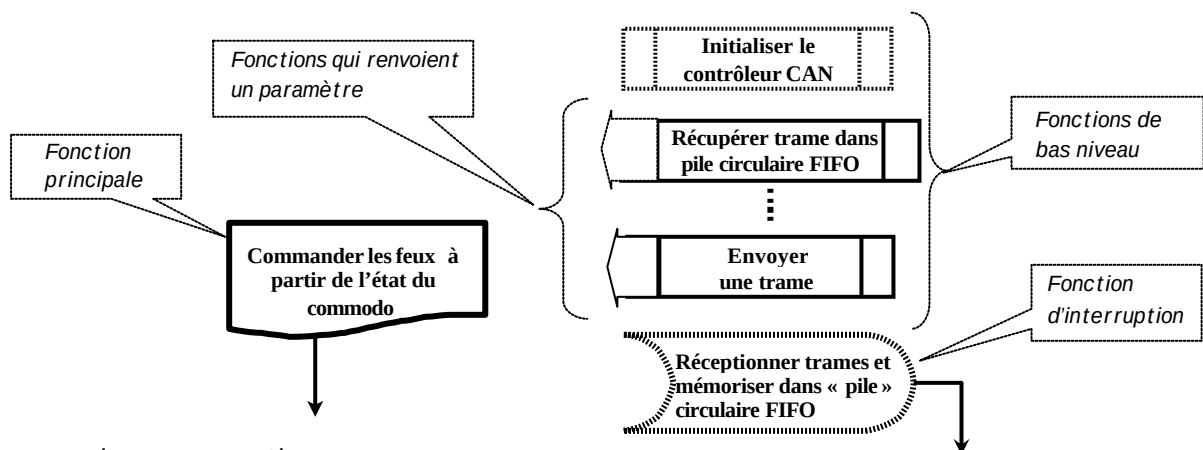
Structuration générale

Un programme d 'application sera composé d'une fonction principale définissant le fonctionnel de l'application (fonction globale), d'un certain nombre de fonctions de bas niveau et éventuellement de fonctions d'interruption.

En ce qui concerne l'application les fonctions de bas niveau font appel aux registres du contrôleur CAN (le SJA 1000) implémenté sur la carte industrielle d'interface CAN au format PC 104, elle-même "pluguée" sur le bus au format PC104 du contrôleur général "EID210".

Exemple : Soit à réaliser la fonction globale suivante :

« Commander les feux avant du véhicule en fonction de l'état du commodo de commande des feux »



Langage de programmation :

Toutes ou parties des différentes fonctions peuvent être écrites soit en assembleur, soit en « C » Dans l'exemple présenté ci-après, le langage choisi est le « C ».

Dans l'environnement de développement du contrôleur EID210 il est possible de mettre en œuvre le moniteur multi-tâches temps réel MTR86.

Outils de programmation :

Environnement de développement de l'EID210 intégrant un cross compilateur-assembleur-linqueur pour micro-processeur de la famille Motorola CPU32.

Les structures de données

1- Les structures de données pour le SJA1000

Elles sont définies pour accéder plus simplement aux différents registres internes au contrôleur SJA1000 de la carte industrielle CAN/PC104.

```

/*****
 * Structure pour l'identificateur *
 * en mode standard *
 *****/
typedef union
{
struct
{
    unsigned short Ident:11;
    unsigned short rtr:1;
    unsigned short nul:4;
} identificateur;

struct
{
    unsigned char ident1;
    unsigned char ident2;
} registre;
    unsigned short valeur;
} ident_standard;

```

Table 35 RX identifier 1 (SFF); CAN address 17; note 1

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID.28	ID.27	ID.26	ID.25	ID.24	ID.23	ID.22	ID.21

Note

1. ID.X means identifier bit X.

Table 36 RX identifier 2 (SFF); CAN address 18; note 1

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID.20	ID.19	ID.18	RTR ⁽²⁾	0	0	0	0

Notes

1. ID.X means identifier bit X.
2. Remote transmission request.

Voir page 43 « DATA SHEET » SJA1000

```

/*****
 * Structure pour l'identificateur *
 * en mode étendu *
 *****/
typedef union
{
struct
{
    unsigned long ident:29;
    unsigned long rtr:1;
    unsigned long x:2;
} identificateur;

struct
{
    unsigned char ident1;
    unsigned char ident2;
    unsigned char ident3;
    unsigned char ident4;
} registre;
    unsigned long valeur;
} ident_extend;

```

Table 38 RX identifier 1 (EFF); CAN address 17; note 1

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID.28	ID.27	ID.26	ID.25	ID.24	ID.23	ID.22	ID.21

Note

1. ID.X means identifier bit X.

Table 39 RX identifier 2 (EFF); CAN address 18; note 1

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID.20	ID.19	ID.18	ID.17	ID.16	ID.15	ID.14	ID.13

Note

1. ID.X means identifier bit X.

Table 40 RX identifier 3 (EFF); CAN address 19; note 1

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID.12	ID.11	ID.10	ID.9	ID.8	ID.7	ID.6	ID.5

Note

1. ID.X means identifier bit X.

Table 41 RX identifier 4 (EFF); can address 20; note 1

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
ID.4	ID.3	ID.2	ID.1	ID.0	RTR ⁽²⁾	0	0

Notes

1. ID.X means identifier bit X.
2. Remote transmission request.

Voir page 43 et 44 « DATA SHEET » SJA1000

```

/* *****
 * Structure pour le registre
 * d'information de frame
 * *****
typedef union
{
    struct
    {
        unsigned char extend:1;
        unsigned char rtr:1;
        unsigned char nul:2;
        unsigned char dlc:4;
    } champ;
    unsigned char registre;
} tr_info;

```

Table 37 RX frame information (EFF); CAN address 16

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
FF ⁽¹⁾	RTR ⁽²⁾	0	0	DLC.3 ⁽³⁾	DLC.2 ⁽³⁾	DLC.1 ⁽³⁾	DLC.0 ⁽³⁾

Notes

1. Frame format.
2. Remote transmission request.
3. Data length code bit.

Voir page 43 « DATA SHEET » SJA1000

```

/* *****
 * Structure pour la trame elle-même
 * *****
/
typedef struct
{
    tr_info frame_info;
    union
    {
        {
            ident_standard standard;
            ident_extend extend;
        } ident;
        unsigned char data[8];
    } Trame;
}

```

CAN address	16	TX frame information
	17	TX identifier 1
	18	TX identifier 2
	19	TX data byte 1
	20	TX data byte 2
	21	TX data byte 3
	22	TX data byte 4
	23	TX data byte 5
	24	TX data byte 6
	25	TX data byte 7
	26	TX data byte 8
	27	unused
	28	unused

CAN address	16	TX frame information
	17	TX identifier 1
	18	TX identifier 2
	19	TX identifier 3
	20	TX identifier 4
	21	TX data byte 1
	22	TX data byte 2
	23	TX data byte 3
	24	TX data byte 4
25	TX data byte 5	
26	TX data byte 6	
27	TX data byte 7	
28	TX data byte 8	

MKG621

a. Standard frame format.

b. Extended frame format.

Voir page 39 « DATA SHEET » SJA1000

2- Les structures de données pour l'application

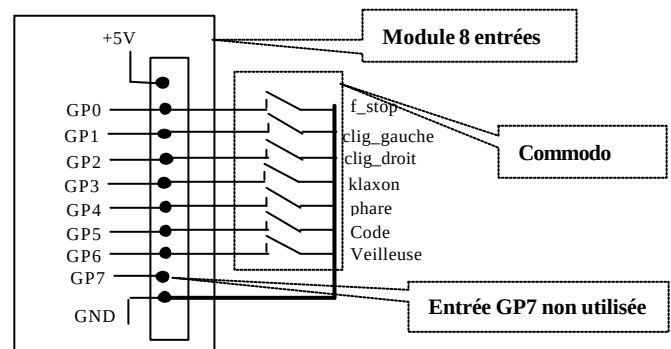
Elles permettent de faciliter l'écriture du programme "application" en facilitant l'accès aux éléments matériels de l'application.

```

/* *****
 * Structure pour le port d'entree
 * COMMODO
 * *****
typedef union
{
    struct
    {
        unsigned char nul:1;
        unsigned char veilleuse:1;
        unsigned char code:1;
        unsigned char phare:1;
        unsigned char klaxon:1;
        unsigned char clig_droit:1;
        unsigned char clig_gauche:1;
        unsigned char f_stop:1;
    } commande;
    unsigned char valeur;
} COMMODO;

```

Le commodo est connecté à un module 8 entrées. Lorsque l'on active une fonction, cela revient, d'un point de vue électrique, à connecter une entrée à 0V. (état 0)



Aux habitués des structures dans le monde "PC", dans le monde "68000" l'ordre de repérage des bits est inverse.

Grâce à cette structure, on accède aux 7 entrées du commodo par la variable `COMMODO.valeur` et pour une fonction parmi les 7 à une variable `COMMODO.commande.code` (entrée code pour exemple).

La fonction d'initialisation du contrôleur CAN « SJA1000 »

On choisit de faire fonctionner le SJA1000 en mode « PELICAN », ce qui permet les échanges en mode CAN2.0A (mode standard) et CAN2.0B (Mode étendu)

```

/*****
* Initialisation du SJA1000 en mode Pelican
*****/

```

```

void init_sja1000_peli_acc (char acc_code0,
                             char acc_code1,
                             char acc_code2,
                             char acc_code3,
                             char acc_mask0,
                             char acc_mask1,
                             char acc_mask2,
                             char acc_mask3)
{
    // On passe en RESET MODE (bit 0)
    // Il faut attendre qu'il y soit !
    while (!(MODE & 0x01)) MODE=0x01;

    // Mode Pelican (bit 7)
    CLOCK_DIVIDER=0xC0;

    // On désactive les interruptions
    INTERRUPT_ENABLE=0x00;
    // On définit les codes pour filtrage et les masques
    ACCEPT_CODE0=acc_code0;
    ACCEPT_CODE1=acc_code1;
    ACCEPT_CODE2=acc_code2;
    ACCEPT_CODE3=acc_code3;
    ACCEPT_MASK0=acc_mask0;
    ACCEPT_MASK1=acc_mask1;
    ACCEPT_MASK2=acc_mask2;
    ACCEPT_MASK3=acc_mask3;
    // On configure la vitesse de transmission 100 kbit/s
    BUS_TIMING_0=0xc4; // 1 1 0 0 0 1 0 0
    BUS_TIMING_1=0x3a; // 0 0 1 1 1 0 1 0
    // On définit le mode de TXD et RXD
    // 11011011
    // Push-pulls, "polarité zero", clock-output mode
    OUTPUT_CONTROL=0x1A;
    /* On définit le mode de fonctionnement
    Bit 0 = 0: Sortie du mode RESET
    Bit 1 = 0: Pas le mode Listen Only
    (donne des ACK)
    Bit 2 = 0: Pas le mode Self Test
    Bit 3 = 1: Acceptance filter mode = single
    et il faut attendre la sortie du mode RESET */
    while (MODE & 0x01) MODE=0x08;
}
// Fin de la fonction d'initialisation

```

Calcul de la vitesse de transmission:

La fréquence du quartz sur la carte est 16 Mhz

On a choisi BRP = 4

$$\rightarrow t_{scl} = 2 \cdot (4+1) \cdot t_{CLK} = 0.625 \mu s$$

On a choisi

TSEG1 = 0x0A = 10 et TSEG2 = 3

D'après « DATA SHEET »

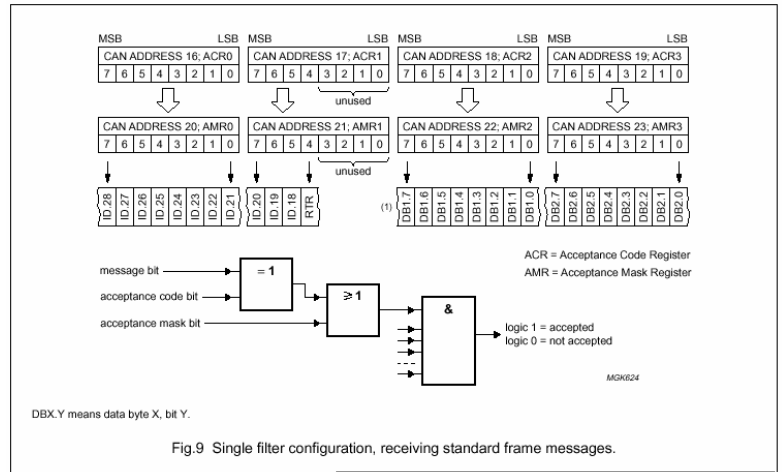
$$\rightarrow t_{SEG1} = 11 \cdot t_{scl} \text{ et } t_{SEG2} = 4 \cdot t_{scl}$$

En définitive :

$$t_{bit} = 16 \cdot t_{scl} = 16 \cdot 0.625 \mu s = 10 \mu s$$

Soit une vitesse de $1/t_{bit} = 100 \text{ Kbit/s}$

Pour le filtrage et le masquage des messages



Voir page 45 « DATA SHEET » SJA1000

Pour définir la vitesse de transmission

Table 44 Bit interpretation of bus timing register 0 (BTR0); CAN address 6

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
SJW.1	SJW.0	BRP.5	BRP.4	BRP.3	BRP.2	BRP.1	BRP.0

6.5.1.1 Baud Rate Prescaler (BRP)

The period of the CAN system clock t_{scl} is programmable and determines the individual bit timing. The CAN system clock is calculated using the following equation:

$$t_{scl} = 2 \times t_{CLK} \times (32 \times BRP.5 + 16 \times BRP.4 + 8 \times BRP.3 + 4 \times BRP.2 + 2 \times BRP.1 + BRP.0 + 1)$$

$$\text{where } t_{CLK} = \text{time period of the XTAL frequency} = \frac{1}{f_{XTAL}}$$

Table 45 Bit interpretation of bus timing register 1 (BTR1); CAN address 7

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
SAM	TSEG2.2	TSEG2.1	TSEG2.0	TSEG1.3	TSEG1.2	TSEG1.1	TSEG1.0

6.5.2.1 Sampling (SAM)

BIT	VALUE	FUNCTION
SAM	1	triple; the bus is sampled three times; recommended for low/medium speed buses (class A and B) where filtering spikes on the bus line is beneficial
	0	single; the bus is sampled once; recommended for high speed buses (SAE class C)

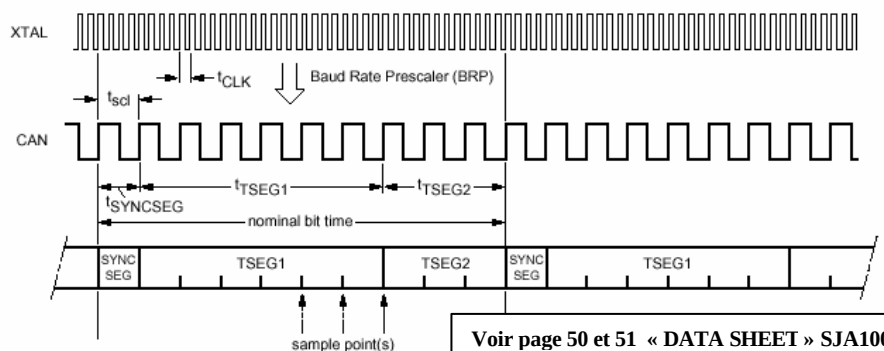
6.5.2.2 Time Segment 1 (TSEG1) and Time Segment 2 (TSEG2)

TSEG1 and TSEG2 determine the number of clock cycles per bit period and the location of the sample point, where:

$$t_{SYNCSEG} = 1 \times t_{scl}$$

$$t_{TSEG1} = t_{scl} \times (8 \times TSEG1.3 + 4 \times TSEG1.2 + 2 \times TSEG1.1 + TSEG1.0 + 1)$$

$$t_{TSEG2} = t_{scl} \times (4 \times TSEG2.2 + 2 \times TSEG2.1 + TSEG2.0 + 1)$$



Voir page 50 et 51 « DATA SHEET » SJA1000

La fonction d'émission d'un message

Pour émettre un message sur le bus CAN, il est indispensable de vérifier au préalable si le transmetteur n'est pas occupé (en train d'émettre une trame) et que le « buffer » d'émission est disponible (il n'est pas plein).

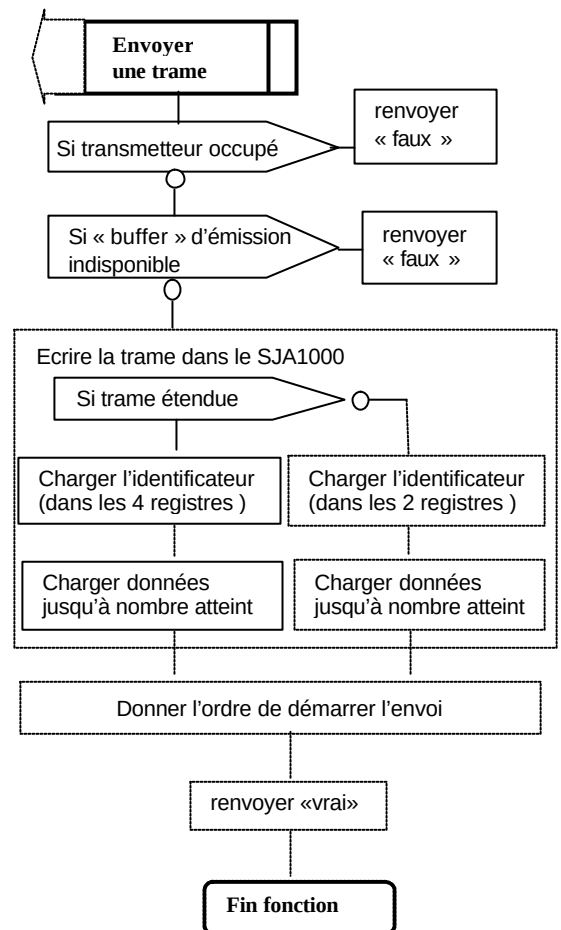
Si l'une des deux conditions n'est pas réunie, la trame n'est pas envoyée et la fonction retourne « faux » (information de type « char »). Dans ce cas, c'est au programmeur de définir dans la fonction principale ce qu'il doit faire (par exemple, réitérer la demande un certain nombre de fois, à intervalles de temps réguliers, et envoyer au superviseur un message d'alerte si le nombre de tentatives infructueuses a été atteint).

Si ces deux conditions sont réunies, il faut alors charger les informations constituant le message dans les différents registres du contrôleur CAN SJA1000, grâce aux structures de données associées. Puis donner l'ordre de démarrer la transmission. La fonction se termine par l'envoi de « vrai » (information de type « char »).

```

/*****
*      Fonction « ENVOYER UNE TRAME »
*****/
char Send_Traine(Frame frame)
{
/* declaration des variables locales */
unsigned char i;
/* Si transmetteur occupé alors renvoyer faux */
if ((STATUS & TRANSMIT_STATUS) == 0) return faux;
/* Si buffer d'émission n'est pas accessible alors renvoyer faux */
if ((STATUS & TRANSMIT_BUFFER_STATUS) == 0) return faux;
/* Ecrire la trame dans le SJA1000 */
TX_FRAME_INFO=frame.frame_info.registre;
if (frame.frame_info.champ.extend==1)
{ // trame étendue
TX_ID_1_E=frame.ident.extend.registre.ident1;
TX_ID_2_E=frame.ident.extend.registre.ident2;
TX_ID_3_E=frame.ident.extend.registre.ident3;
TX_ID_4_E=frame.ident.extend.registre.ident4;
if (frame.frame_info.champ.rtr==0)
{
for(i=0;i<frame.frame_info.champ.dlc;i++)
*(TX_DATA_E+i)=frame.data[i];
}
}
else
{ // trame standard
TX_ID_1_S=frame.ident.standard.registre.ident1;
TX_ID_2_S=frame.ident.standard.registre.ident2;
if (frame.frame_info.champ.rtr==0)
{
for(i=0;i<frame.frame_info.champ.dlc;i++)
*(TX_DATA_S+i)=frame.data[i];
}
}
// Donner l'ordre de démarrer l'envoi (bit 0)
COMMAND=0x01;
} /* Fin fonction */

```

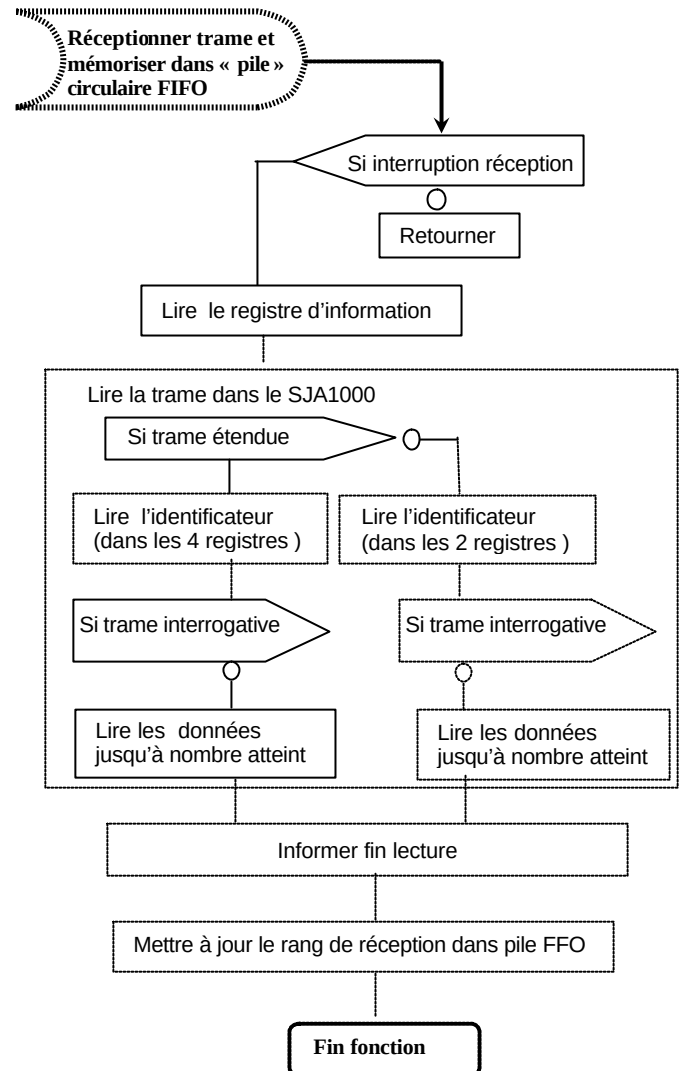


La fonction d'interruption permettant la réception d'une trame

```

/*****
 * PROGRAMME D'INTERRUPTION permettant de
 * réceptionner les trames acceptées et de les mémoriser
 * dans la « pile » circulaire de type FIFO
 *****/
void irq_can()
{
    // Déclaration variable locale
    int i;
    // Tester si c'est bien une interruption de réception
    if (INTERRUPT&0x01) // Bit 0 de l'octet INTERRUPT
    // Si l'interruption vient d'une trame reçue
    /*****
    {
        // Lire le registre d'information associé
        rx_mes[rang_recept].trame_info.registre=RX_FRAME_INFO;
        // Tester si le bit indicateur si trame étendue
        if (rx_mes[rang_recept].trame_info.champ.extend==1)
        {
            // Si c'est une trame étendue
            // *****
            // Lire les registres contenant l'identificateur étendu
            rx_mes[rang_recept].ident.extend.registre.ident1=RX_ID_1_E;
            rx_mes[rang_recept].ident.extend.registre.ident2=RX_ID_2_E;
            rx_mes[rang_recept].ident.extend.registre.ident3=RX_ID_3_E;
            rx_mes[rang_recept].ident.extend.registre.ident4=RX_ID_4_E;
            // Tester si ce n'est pas une trame interrogative
            if (rx_mes[rang_recept].trame_info.champ.rtr==0)
            // C'est une trame de données donc on les lit
            {
                for(i=0;i<rx_mes[rang_recept].trame_info.champ.dlc;i++)
                rx_mes[rang_recept].data[i]=*(RX_DATA_E+i);
            }
        }
        else
        { // Si c'est une trame standard
            // *****
            // Lire les registres contenant l'identificateur standard
            rx_mes[rang_recept].ident.standard.registre.ident1=RX_ID_1_S;
            rx_mes[rang_recept].ident.standard.registre.ident2=RX_ID_2_S;
            // Tester si ce n'est pas une trame interrogative
            if (rx_mes[rang_recept].trame_info.champ.rtr==0)
            // C'est une trame de données donc on les lit
            {
                for(i=0;i<rx_mes[rang_recept].trame_info.champ.dlc;i++)
                rx_mes[rang_recept].data[i]=*(RX_DATA_S+i);
            }
        }
        // Release receive buffer */
        COMMAND=0x04;
        // Faire évoluer le rang de dans la pile FIFO circulaire
        rang_recept++;
        if (rang_recept==MAX_MESSAGE)
            rang_recept=0;
    }
}

```



Rôle du bit 0 du INTERRUPT register (voir page 31&32 DOC SJA1000)

IR.0	RI	Receive Interrupt; note 2	1	set; this bit is set while the receive FIFO is not empty and the RIE bit is set within the interrupt enable register
			0	reset; no more message is available within the RXFIFO

2. The behaviour of this bit is equivalent to that of the receive buffer status bit with the exception, that RI depends on the corresponding interrupt enable bit (RIE). So the receive interrupt bit is not cleared upon a read access to the interrupt register. Giving the command 'release receive buffer' will clear RI temporarily. If there is another message available within the FIFO after the release command, RI is set again. Otherwise RI remains cleared.

Rôle du bit 2 du COMMAND register (voir page 28 DOC SJA1000)

CMR.2	RRB	Release Receive Buffer; note 4	1	released; the receive buffer, representing the message memory space in the RXFIFO is released
			0	– (no action)

4. After reading the contents of the receive buffer, the CPU can release this memory space in the RXFIFO by setting the release receive buffer bit to logic 1. This may result in another message becoming immediately available within the receive buffer. If there is no other message available, the receive interrupt bit is reset.

La fonction de récupération d'un message dans la pile FIFO

Lorsqu'un message est accepté et récupéré par le contrôleur CAN (SJA1000), il y a génération d'une interruption. Dans la fonction d'interruption, ce message est stocké dans une pile circulaire de type FIFO (First In First Out) → premier entré, premier sorti.

Données déclarées en global :

Deux variables globales permettent de gérer cette pile :

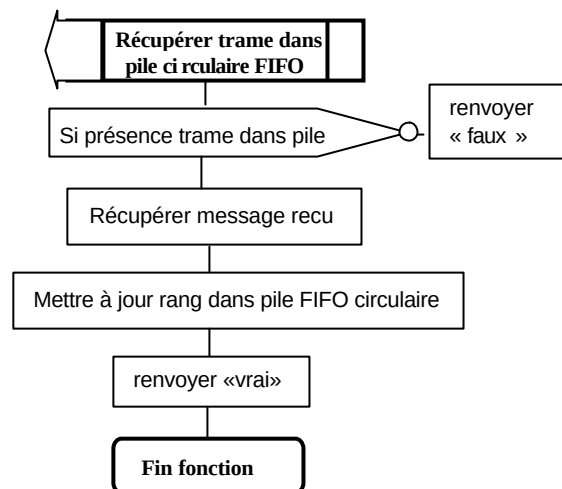
→ l'une donne le rang du dernier message reçu et placé dans la pile (rang_recept),

→ l'autre donne le rang du premier message reçu restant dans la pile (rang_lect).

La taille de la pile circulaire FIFO est définie par la constante Nb_MAX_MESSAGE.

La pile circulaire est en fait un tableau de messages de taille Nb_MAX_MESSAGE.

```
char Recuperer_trame(Trame *message_recu)
{
  /* Si présence trame dans pile circulaire FIFO */
  if (rang_lect!=rang_recept)
  {
    /* Alors récupérer message reçu */
    *message_recu=rx_mes[rang_lect];
    rang_lect++;
    if (rang_lect==Nb_MAX_MESSAGE)
      rang_lect=0;
    return vrai;
  }
  return faux;
}
```



Fonction principale

La fonction principale d'un programme pour micro-processeur comporte obligatoirement les deux parties suivantes :

- une partie destinée aux initialisations
- une boucle principale qui tourne à l'infini → `do{.....} while(1);` ou encore `while(1) {.....}`

Dans le cas de l'environnement de développement intégré du contrôleur EID210, si on sort de la boucle principale on redonne la main au moniteur.

Dans le cas de l'application présentée, le module 8 entrées sur lequel a été configuré, pour envoyer sur le bus CAN, un message donnant l'état du port et ce à intervalles de temps réguliers (activation du "scheduler"). Pour récupérer l'état des entrées commodo il suffit donc de se mettre à l'écoute des trames reçues.

```

*****
* FONCTION PRINCIPALE : Commandes des feux avants sur automobile multiplexée *
*****
* Auteurs: ROHOU Jean Luc HANS Thierry Date de création: 16/11/02 *
*****
main()
{
// Définition des variables
COMMODE commodo;
Trame message;
Trame feux_ag;
Trame feux_ad;
rang_recept=0;
rang_lect=0;
/* INITIALISATIONS */
*****/
// L'écran du moniteur associé au contrôleur EID210
clrscr();
// trame pour le feux arrière gauche
feux_ag.trame_info.registre=0x00;
feux_ag.trame_info.champ.dlc=0x03;
feux_ag.ident.standard.identificateur.ident=0x010;
feux_ag.data[0]=0x1e;
feux_ag.data[1]=0x0f;
feux_ag.data[2]=0x00;
// trame pour le feux arrière droit
feux_ad.trame_info.registre=0x00;
feux_ad.trame_info.champ.dlc=0x03;
feux_ad.ident.standard.identificateur.ident=0x020;
feux_ad.data[0]=0x1e;
feux_ad.data[1]=0x0f;
feux_ad.data[2]=0x00;
/* Initialiser le contrôleur CAN SJA1000 de la carte PC104 */
init_sja1000_peli_acc (0x03,0x00,0xFF,0xFF,0xFF,0xFF,0xFF,0xFF);
/* Initialiser les interruptions afin de rendre actif le programme d'interruption de récupération des trames reçues */
SetVect(29,&irq_can);
INTERRUPT_ENABLE=0x01;
VALID_IRQ.valeur=0x20;
* BOUCLE PRINCIPALE *
*****/
while(1)
{
if (1==Recuperer_trame(&message)) // Si une trame a été acceptée et récupérée grâce au programme d'interruption
{
if (message.trame_info.champ.extend==0 && message.ident.standard.identificateur.ident==0x0080)
{
commodo.valeur=~message.data[1];
if (commodo.commande.veilleuse==1)
{
feux_ad.data[2]=0x01; // Préparer pour activer veilleuse
feux_ag.data[2]=0x01;
}
else
{
feux_ad.data[2]&=0xFE;
feux_ag.data[2]&=0xFE;
}
if (commodo.commande.f_stop==1) // Préparer pour activer feux stop
{
feux_ad.data[2]=0x02;
feux_ag.data[2]=0x02;
}
else
{
feux_ad.data[2]&=0xFD;
feux_ag.data[2]&=0xFD;
}
if (commodo.commande.klaxon==1) feux_ad.data[2]=0x08; // Préparer pour activer Klaxon
else feux_ad.data[2]&=0xF7;
if (commodo.commande.clign_droit==1) feux_ad.data[2]^=0x04; // Préparer pour activer clign_droit
else feux_ad.data[2]&=0xFB;
if (commodo.commande.clign_gauche==1) feux_ag.data[2]^=0x04; // Préparer pour activer clign_gauche
else feux_ag.data[2]&=0xFB;
Send_Trame(feux_ad); // Envoi de la trame pour activer le bloc optique avant droit
Send_Trame(feux_ag); // Envoi de la trame pour activer le bloc optique avant gauche
}
}
}
} // Fin de la fonction principale

```

5.3.2 Le logiciel «Supervision»

Intérêt didactique :

Etre capable de définir les structures de données permettant la communication entre le contrôleur d'application EID210 et le PC destiné à la supervision.

Etre capable de configurer et de réaliser les logiciels de communication via un port U.S.B. (Universal Serial Bus) :

- ® le logiciel côté contrôleur EID 210,
- ® le logiciel côté PC.

Outils de programmation

- Environnement de développement de l'EID210,
- Environnement Visual C, Visual Basic ou Borland C++ builder.

5.3.3 Le logiciel «Communication»

Intérêt didactique :

Etre capable de définir les structures de données permettant l'affichage d'informations pertinentes sur une page WEB dynamique (serveur WEB).

Connaître la programmation H.T.M.L. (Hypertext Markup Language).

Connaître le fonctionnement et le paramétrage d'une pile TCP/IP (Transport Control Protocol/ Internet Protocol).

Outils de programmation

- Environnement de développement de l'EID210,

5.3.4 Le logiciel « Analyse trames CAN»

Intérêt didactique :

Etre capable de comprendre et d'utiliser des logiciels spécifiques fournis par le constructeur de la carte d'interface bus CAN « plugguée » dans le PC.

Etre capable d'installer, de configurer et de mettre en œuvre une carte d'interface bus CAN au format PC.

Etre capable d'installer, de configurer et de mettre en œuvre le logiciel associé à une carte d'interface bus CAN au format PC.

Outils de programmation

- Environnement Visual C, Visual Basic ou Borland C++ builder.

6 LIENS INTERNETS "SYMPA" ET UTILES

www.cia-can.org

www.multimania.com/xcotton/electron/coursetdocs

www.microchip.com

www.philips.com

Pour nous contacter : [**ge@didalab.fr**](mailto:ge@didalab.fr)